Василенко Е.Р. 090303-ПИа-о24

Лабораторная работа №14
Разработка REST API на Python с использованием FastAPI

1. Цель работы

Получить практические навыки разработки современных REST API на Python с использованием фреймворка FastAPI: создавать HTTP-эндпоинты, работать с методами GET, POST, PUT, PATCH, DELETE, параметрами URL и query-параметрами, принимать и возвращать JSON, использовать Pydantic для валидации данных, корректно применять HTTP-коды состояния, организовывать обработку ошибок, использовать автоматическую документацию OpenAPI, разделять API на слои Router → Service → Repository → Database, применять Dependency Injection, реализовать фильтрацию, сортировку, поиск и пагинацию, подключить базу данных и кэширование, а также написать автоматические тесты API.

2. Краткие теоретические сведения

FastAPI — современный Python-фреймворк для разработки API. Простейшее приложение состоит из экземпляра FastAPI и декораторов: @app.get("/products") определяет эндпоинт — комбинацию HTTP-метода и URL. FastAPI автоматически преобразует path-параметры (/products/{id} → int), разбирает query-параметры фильтрации и пагинации, валидирует тело запроса через Pydantic и формирует интерактивную документацию OpenAPI (Swagger UI на /docs и ReDoc на /redoc).

Pydantic-модели описывают структуру данных: RoomCreate (тело запроса), RoomUpdate (частичное изменение, все поля необязательны), RoomResponse (ответ) и RoomListResponse. Поля задаются с ограничениями через Field: price > 0, capacity >= 1. FastAPI автоматически проверяет типы и наличие полей и возвращает 422 Validation Error при несоответствии. Response model отделяет внутренние данные от того, что получает клиент.

Корректные HTTP-коды: 200 OK, 201 Created, 204 No Content, 404 Not Found, 409 Conflict, 422 Validation Error, 500 Internal Server Error. Dependency

Injection (Depends) позволяет передавать в обработчики сервисы и сессии базы данных, отделяя HTTP-слой от бизнес-логики. FastAPI поддерживает синхронные (def) и асинхронные (async def) обработчики; асинхронность эффективна при ожидании операций ввода-вывода (сеть, БД, Redis), но не ускоряет CPU-bound вычисления.

3. Индивидуальное задание

Вариант 5 — предметная область «Отель», сущность Room. Поля: номер, тип, этаж, вместимость, цена, доступность, удобства. Требуется реализовать REST API с полным CRUD (GET, GET/{id}, POST, PUT, PATCH, DELETE), поиском, фильтрацией (?type, ?min_price, ?max_price, ?available), сортировкой (sort, order) и пагинацией (skip, limit), статистикой, Pydantic-схемами, SQLAlchemy-хранилищем, кэшированием чтения через Redis (cache-aside, TTL, инвалидация), минимум одним асинхронным эндпоинтом, автоматической документацией OpenAPI и не менее 15 автоматических тестов.

4. Архитектура приложения

Приложение построено по многослойной схеме: Client → Router → Service → Repository → Database, с Redis-кэшем на уровне чтения по ID. HTTP-маршруты только разбирают запросы, бизнес-логика находится в сервисе, доступ к данным — в репозитории, а хранилище изолировано за SQLAlchemy. Схема приведена на рисунке 14.1.

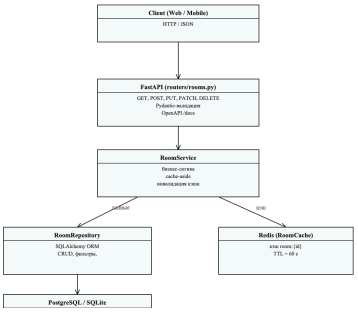


Рисунок 14.1 – Архитектура REST API

```
lab_14/
├── app/
│   ├── main.py                # приложение, middleware-логирование, seed-данные
│   ├── database.py            # Engine, Session, Dependency Injection get_db
│   ├── models/room.py         # ORM-модель Room
│   ├── schemas/room.py        # Pydantic: RoomCreate, RoomUpdate, RoomResponse…
│   ├── repositories/room.py   # CRUD, фильтрация, сортировка, пагинация,
статистика
│   ├── services/room.py       # бизнес-логика и кэш (cache-aside)
│   └── routers/rooms.py       # HTTP-эндпоинты /rooms
├── tests/test_api.py          # 22 теста API
├── requirements.txt
└── README.md
```

Назначение слоёв: Router работает с HTTP и вызывает сервис через Depends; Service содержит бизнес-логику и проверки; Repository выполняет запросы через SQLAlchemy ORM; Schema (Pydantic) описывает входные и выходные данные API; Model представляет структуру таблицы. Такое разделение позволяет тестировать каждый уровень отдельно и соответствует принципу separation of concerns.

5. Проектирование API

Таблица эндпоинтов API приведена ниже.

Метод	URL	Назначение	Код
GET	/rooms	список с фильтрацией, сортировкой, пагинацией	200
GET	/rooms/{id}	номер по идентификатору	200/404
POST	/rooms	создание номера	201/409/422
PUT	/rooms/{id}	полное изменение	200/404
PATCH	/rooms/{id}	частичное изменение	200/404
DELETE	/rooms/{id}	удаление номера	204/404
GET	/rooms/search	поиск по номеру или типу (?q=)	200
GET	/rooms/statistics	статистика	200

QUERY-параметры списка: room_type, min_price, max_price, available (фильтрация), sort (price/number/capacity) и order (asc/desc) — сортировка, skip и limit — пагинация. Фильтрация по цене поддерживает диапазон, доступность — булево значение. Спецификация OpenAPI автоматически генерируется FastAPI и доступна по адресу /openapi.json; интерактивная документация — на /docs (Swagger UI) и /redoc (ReDoc).

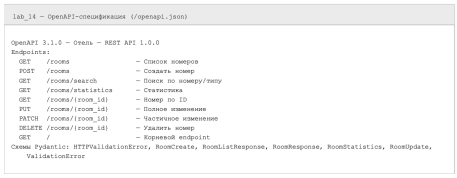


Рисунок 14.2 – OpenAPI-спецификация API

6. Pydantic-схемы

Файл: app/schemas/room.py

```
from typing import List, Optional

from pydantic import BaseModel, Field


class RoomBase(BaseModel):
    number: str = Field(min_length=1, max_length=10, description="Номер
комнаты")
    room_type: str = Field(min_length=1, description="Тип: стандартный / люкс /
семейный")
    floor: int = Field(ge=1, le=100, description="Этаж")
    capacity: int = Field(ge=1, description="Вместимость, человек")
    price: float = Field(gt=0, description="Цена за ночь, руб.")
    available: bool = True
    amenities: List[str] = Field(default_factory=list, description="Удобства")


class RoomCreate(RoomBase):
    pass


class RoomUpdate(BaseModel):
    number: Optional[str] = Field(default=None, min_length=1, max_length=10)
    room_type: Optional[str] = Field(default=None, min_length=1)
    floor: Optional[int] = Field(default=None, ge=1, le=100)
    capacity: Optional[int] = Field(default=None, ge=1)
    price: Optional[float] = Field(default=None, gt=0)
    available: Optional[bool] = None
    amenities: Optional[List[str]] = None


class RoomResponse(RoomBase):
    id: int

    class Config:
        from_attributes = True


class RoomListResponse(BaseModel):
    items: List[RoomResponse]
    total: int
    skip: int
    limit: int


class RoomStatistics(BaseModel):
    total: int
    available_count: int
    average_price: float
    min_price: float
```

```
    max_price: float
```

Scехемы разделены по назначению: RoomCreate используется в POST и PUT (обязательные поля с ограничениями), RoomUpdate — в PATCH (все поля необязательны, изменяются только переданные), RoomResponse формирует ответ и включает id, RoomListResponse оборачивает список с метаданными пагинации, RoomStatistics описывает результат статистики. Response model в декораторах гарантирует, что клиент получит только описанные поля.

7. Работа с базой данных (SQLAlchemy)

ORM-модель Room описана в стиле SQLAlchemy 2.x: типизированные столбцы Mapped/mapped_column, уникальный номер, обязательные поля. Репозиторий изолирует SQL-запросы: CRUD-операции, фильтрацию и сортировку через select().where().order_by(), пагинацию через offset/limit и статистику через агрегатные функции count, avg, min, max.

Файл: app/models/room.py

```
from sqlalchemy import String, Integer, Float, Text
from sqlalchemy.orm import Mapped, mapped_column

from app.database import Base


class Room(Base):
    """ORM-модель гостиничного номера."""

    __tablename__ = "rooms"

    id: Mapped[int] = mapped_column(primary_key=True)
    number: Mapped[str] = mapped_column(String(10), unique=True, nullable=False)
    room_type: Mapped[str] = mapped_column(String(20), nullable=False)
    floor: Mapped[int] = mapped_column(Integer, nullable=False)
    capacity: Mapped[int] = mapped_column(Integer, nullable=False)
    price: Mapped[float] = mapped_column(Float, nullable=False)
    available: Mapped[bool] = mapped_column(default=True, nullable=False)
    amenities: Mapped[str] = mapped_column(Text, default="", nullable=False)

    def __repr__(self) -> str:
        return f"Room(id={self.id}, number={self.number!r}, price={self.price})"
```

Файл: app/repositories/room.py (фрагмент)

```
    def list(self, room_type: Optional[str] = None, min_price: Optional[float] =
None,
             max_price: Optional[float] = None, available: Optional[bool] =
None,
             sort: str = "price", order: str = "asc",
             skip: int = 0, limit: int = 20) -> Tuple[List[Room], int]:
        stmt = select(Room)
        count_stmt = select(func.count(Room.id))
        if room_type:
            stmt = stmt.where(Room.room_type == room_type)
            count_stmt = count_stmt.where(Room.room_type == room_type)
        if min_price is not None:
            stmt = stmt.where(Room.price >= min_price)
```

```
            count_stmt = count_stmt.where(Room.price >= min_price)
        if max_price is not None:
            stmt = stmt.where(Room.price <= max_price)
            count_stmt = count_stmt.where(Room.price <= max_price)
        if available is not None:
            stmt = stmt.where(Room.available == available)
            count_stmt = count_stmt.where(Room.available == available)

        column = {"price": Room.price, "number": Room.number, "capacity":
Room.capacity}.get(sort, Room.price)
        stmt = stmt.order_by(column.asc() if order == "asc" else column.desc())
        stmt = stmt.offset(skip).limit(limit)

        return list(self._db.scalars(stmt)), int(self._db.scalar(count_stmt) or
0)

    def search(self, query: str, limit: int = 20) -> List[Room]:
        pattern = f"%{query}%"
        stmt = select(Room).where(Room.number.contains(query) |
Room.room_type.contains(query)).limit(limit)
        return list(self._db.scalars(stmt))

    def create(self, number: str, room_type: str, floor: int, capacity: int,
               price: float, available: bool, amenities: List[str]) -> Room:
        room = Room(
```

Сессия SQLAlchemy создаётся на время одного запроса и передаётся через Dependency Injection (get_db) — это стандартный паттерн FastAPI. Строка подключения задаётся переменной окружения DATABASE_URL; по умолчанию используется SQLite, для PostgreSQL достаточно указать postgresql+psycopg://...

8. Кэширование через Redis

Чтение номера по ID реализовано по стратегии cache-aside: сначала проверяется Redis (ключ room:{id}, TTL 60 секунд); при попадании возвращается JSON из кэша, при промахе данные читаются из базы и помещаются в кэш. После PUT, PATCH и DELETE ключ удаляется (инвалидация), поэтому следующий запрос получает свежие данные. Счётчики hits/misses показывают эффективность кэша. При недоступности Redis используется fakeredis, поэтому API работает в любом окружении.

Файл: app/services/room.py (кэш и чтение)

```
        self._client = client
        self._ttl = ttl
        self.hits = 0
        self.misses = 0

    @staticmethod
    def key(room_id: int) -> str:
        return f"room:{room_id}"

    def get(self, room_id: int) -> Optional[dict]:
        raw = self._client.get(self.key(room_id))
        if raw is None:
            self.misses += 1
            return None
```

```
        self.hits += 1
        return json.loads(raw)

    def set(self, room_id: int, data: dict) -> None:
        self._client.set(self.key(room_id), json.dumps(data,
ensure_ascii=False), ex=self._ttl)

    def invalidate(self, room_id: int) -> None:
        self._client.delete(self.key(room_id))

    def ttl(self, room_id: int) -> int:
        return int(self._client.ttl(self.key(room_id)))

    def flush(self) -> None:
        for key in self._client.keys("room:*"):
            self._client.delete(key)
        self.hits = 0
        self.misses = 0

    def stats(self) -> dict:
        return {"hits": self.hits, "misses": self.misses} if self._client is not
None else {}


class RoomService:
    """Бизнес-логика: валидация, CRUD, фильтрация, кэширование."""

    def __init__(self, repository: RoomRepository, cache: Optional[RoomCache] =
None):
        self._repository = repository
        self._cache = cache

    def list_rooms(self, room_type=None, min_price=None, max_price=None,
available=None,
                   sort="price", order="asc", skip=0, limit=20) ->
Tuple[List[dict], int]:
        rooms, total = self._repository.list(
            room_type=room_type, min_price=min_price, max_price=max_price,
```

9. HTTP-эндпоинты

Маршруты не содержат бизнес-логики: каждый обработчик принимает параметры и тело запроса, вызывает метод сервиса через Depends и возвращает схему. Специализированные эндпоинты поиска и статистики объявлены раньше /rooms/{id}, чтобы FastAPI не трактовал их как идентификатор.

Файл: app/routers/rooms.py

```
import logging
from typing import List, Optional

from fastapi import APIRouter, Depends, HTTPException, Query
from sqlalchemy.orm import Session

from app.database import get_db
from app.repositories.room import RoomNotFoundError, RoomRepository
from app.schemas.room import RoomCreate, RoomListResponse, RoomResponse,
RoomStatistics, RoomUpdate
from app.services.room import RoomCache, RoomService

logger = logging.getLogger("hotel_api")

router = APIRouter(prefix="/rooms", tags=["rooms"])


def get_service(db: Session = Depends(get_db)):
    """Dependency Injection: сервис с репозиторием и кэшем."""
```

```
    cache_client, _ = getattr(get_service, "_cache_client", (None, None))
    if cache_client is None:
        from app.services.room import create_cache_client

        cache_client, _ = create_cache_client()
        get_service._cache_client = (cache_client, _)
    return RoomService(RoomRepository(db), RoomCache(cache_client))


@router.get("", response_model=RoomListResponse,
            summary="Список номеров", description="Фильтрация, сортировка и
пагинация")
def list_rooms(
    room_type: Optional[str] = Query(default=None, description="Тип номера"),
    min_price: Optional[float] = Query(default=None, description="Мин. цена"),
    max_price: Optional[float] = Query(default=None, description="Макс. цена"),
    available: Optional[bool] = Query(default=None, description="Только
доступные"),
    sort: str = Query(default="price", description="Поле сортировки: price,
number, capacity"),
    order: str = Query(default="asc", pattern="^(asc|desc)$",
description="Направление"),
    skip: int = Query(default=0, ge=0, description="Сколько пропустить"),
    limit: int = Query(default=20, ge=1, le=100, description="Сколько вернуть"),
    service: RoomService = Depends(get_service),
):
    items, total = service.list_rooms(
        room_type=room_type, min_price=min_price, max_price=max_price,
available=available,
        sort=sort, order=order, skip=skip, limit=limit,
    )
    return {"items": items, "total": total, "skip": skip, "limit": limit}


@router.get("/search", response_model=List[RoomResponse], summary="Поиск по
номеру/типу")
def search_rooms(
    q: str = Query(..., min_length=1, description="Строка поиска"),
    limit: int = Query(default=20, ge=1, le=100),
    service: RoomService = Depends(get_service),
):
    return service.search_rooms(q, limit=limit)


@router.get("/statistics", response_model=RoomStatistics, summary="Статистика")
def room_statistics(service: RoomService = Depends(get_service)):
    return service.statistics()
```

Один эндпоинт реализован в асинхронном стиле (async def root) — FastAPI исполняет его в event loop без блокировки потока. Асинхронность полезна при ожидании ввода-вывода (сеть, БД), но не ускоряет CPU-bound операции.

10. Демонстрация работы

Тест 1. REST API: список номеров, получение по ID, создание (POST 201), частичное изменение (PATCH) и удаление (DELETE 204) — рисунок 14.3.

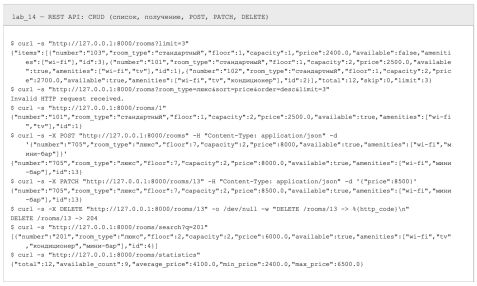


Рисунок 14.3 – REST API: CRUD-операции

Тест 2. Поиск по номеру и статистика (рисунок 14.4).



Рисунок 14.4 – Поиск и статистика

Тест 3. Фильтрация по типу и диапазону цены, сортировка по убыванию, пагинация (рисунок 14.5).

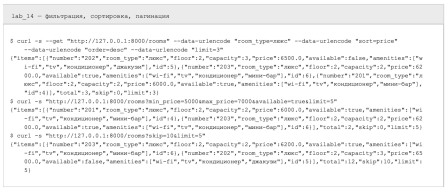


Рисунок 14.5 – Фильтрация, сортировка и пагинация

Тест 4. Обработка ошибок: 404 для отсутствующего номера, 409 для дубликата, 422 для неположительной цены (рисунок 14.6).



Рисунок 14.6 – Обработка ошибок (404, 409, 422)

Тест 5. Журнал сервера: каждый запрос фиксируется middleware с методом, URL, статусом и временем выполнения.

11. Тестирование

Написано 22 автоматических теста pytest с использованием TestClient: список, получение существующего и отсутствующего объекта, создание корректного и некорректного (422), дубликат (409), PUT, PATCH, DELETE, повторное удаление (404), фильтрация по типу и диапазону цен, сортировка, пагинация, поиск, статистика, валидация ошибок, кэш (попадание после первого GET) и инвалидация кэша после PATCH. Тесты используют встроенный TestClient FastAPI и не требуют запущенного сервера. Результат выполнения pytest приведён на рисунке 14.7.

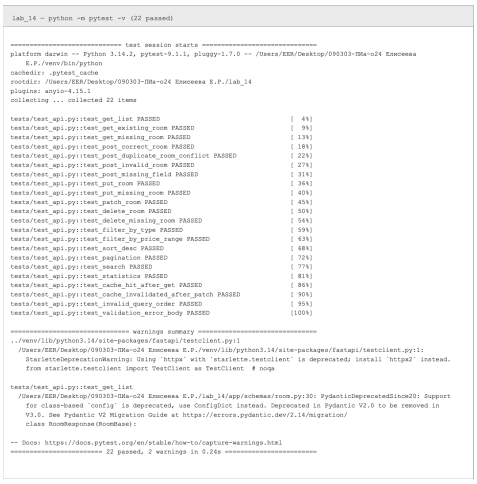


Рисунок 14.7 – Результаты выполнения pytest (22 passed)

12. Вывод

В ходе лабораторной работы разработан полноценный REST API гостиницы на FastAPI: реализованы все обязательные эндпоинты CRUD, поиск, фильтрация, сортировка, пагинация и статистика; данные валидируются Pydantic-схемами с корректными HTTP-кодами; хранилище изолировано за слоями Router → Service → Repository → SQLAlchemy, а чтение по ID кэшируется через Redis по стратегии cache-aside с TTL и инвалидацией.

Автоматическая документация OpenAPI (Swagger UI) даёт возможность другому разработчику изучить и вызвать все эндпоинты без отдельного описания; 22 автоматических теста покрывают CRUD, фильтрацию, сортировку, пагинацию, обработку ошибок и кэширование. Работа объединила знания предыдущих лабораторных — HTTP и REST, SQLAlchemy, Redis — в реальном backend-приложении, где FastAPI отвечает за HTTP-слой, Pydantic — за структуру данных, Service — за бизнес-логику, Repository — за доступ к данным, а PostgreSQL и Redis — за хранение и кэширование.